



Excellence in education for a new generation of Naval Architects and Marine Engineers

SUSTAINABLE SHIP AND SHIPPING 4.0

Erasmus Mundus Joint Master Degree

INDUSTRY 4.0 ENABLING TECHNOLOGIES

Project: *Hybrid ship propulsion system power management and health monitoring*

Ethereum Smart Contract: *Shipowner – Insurance Companies*

Submitted By

Emmanuel Abazuonu

Andres Toala

Md Abidul Islam

Submitted To

Professor Paula Fraga Lamas (Course Coordinator)

Tiago M. Fernandez Carames



University of
Zagreb

TUHH
Technische Universität Hamburg

1. Introduction

Electric propulsion rapidly transforms the maritime industry, replacing traditional diesel engines with cleaner, more efficient electric systems. At the heart of this innovation lies the battery bank, which is the primary power source for the vessel's engine, navigation, and auxiliary systems. These batteries must operate reliably under varying environmental and operational conditions, often in harsh maritime settings. Unlike conventional engines, electric vessels depend entirely on the health of their batteries. A failure in battery performance—due to overheating, overcurrent, fire, or smoke—can lead not only to equipment damage but also to safety hazards and mission failure. Thus, monitoring the thermal and environmental conditions surrounding the batteries becomes critical for ensuring safe and efficient operation. This project, titled "Health Monitoring System for Battery of an Electric Vessel," aims to address this need by developing a smart system that continuously tracks temperature, humidity, smoke levels, flame detection, and rotational speed (RPM) of associated components. By integrating sensors with real-time alerts, local display, and remote MQTT-based reporting, the system provides a proactive safety mechanism for electric vessels, enhancing operational reliability, preventing accidents, and extending battery lifespan.

2. System Overview and Components

This project implements a real-time battery health and safety monitoring system for electric vessels. Using an ESP8266 microcontroller, it reads various sensor values (temperature, humidity, smoke, flame, and RPM) and provides both local display and remote monitoring capabilities via MQTT.

Table 1: List of Components

Component	Type	Function / Job
DHT11 Sensor	Sensor	Measures ambient temperature and humidity
MQ2 Smoke Sensor	Sensor	Detects smoke concentration in the battery environment
Flame Sensor	Sensor	Detects the presence of flame or fire
Hall Effect Sensor	Sensor	Measures RPM (rotational speed) using magnetic pulse detection
Vibration Sensor	Sensor	Monitors vibration levels in the propulsor system (optional in your flowchart)
I2C LCD (16x2)	Output Device	Displays real-time sensor readings locally
Green LED	Actuator	Indicates normal operation (all conditions safe)

Component	Type	Function / Job
Red LED	Actuator	Turns on during alert conditions (e.g., fire, smoke, overheating)
Buzzer	Actuator	Sounds an alarm when a hazard is detected
Relay Module	Actuator	Controls an external system like a cooling fan or extinguisher
ESP8266	Controller	Central processing unit handles sensor input, control logic, and MQTT communication
ESC	Speed Controller	Distributes power from the batteries to each motor and controls rpm

3. Objectives

This project aims to develop a battery health monitoring system for electric vessels with the following objectives:

1. **Ensure Safety:** Detect potentially dangerous conditions such as high smoke levels, flames, or excessive temperature that could indicate battery failure or fire risk. Early detection helps prevent accidents and ensures crew and vessel safety.
2. **Real-Time Monitoring:** Continuously monitor key environmental factors—temperature, humidity, and air quality—as well as the rotational speed (RPM) of connected components to evaluate the motors and system performance in real time.
3. **Local Alerts:** Provide instant feedback through LEDs, a buzzer, and a relay-controlled output to alert personnel during critical situations. These alerts support immediate action, such as cooling or isolation of affected components.
4. **Remote Reporting:** Transmit sensor readings to a remote MQTT broker, allowing data to be monitored externally. This feature supports integration with cloud-based dashboards or vessel monitoring systems for enhanced oversight.
5. **Support Maintenance:** Provide valuable insights into system behaviour that help operators detect early signs of malfunction. This allows scheduled maintenance to be performed before critical failures occur, improving system reliability and extending battery life.

4. Communication Architecture

The system architecture integrates IoT sensors with cloud-based monitoring. Sensor data is collected by ESP8266 gateways and transmitted via the MQTT protocol to a cloud-based Node-RED server. The data is then processed, visualized on dashboards, and, if desired, stored in third-party platforms like Google Cloud for analysis and reporting.

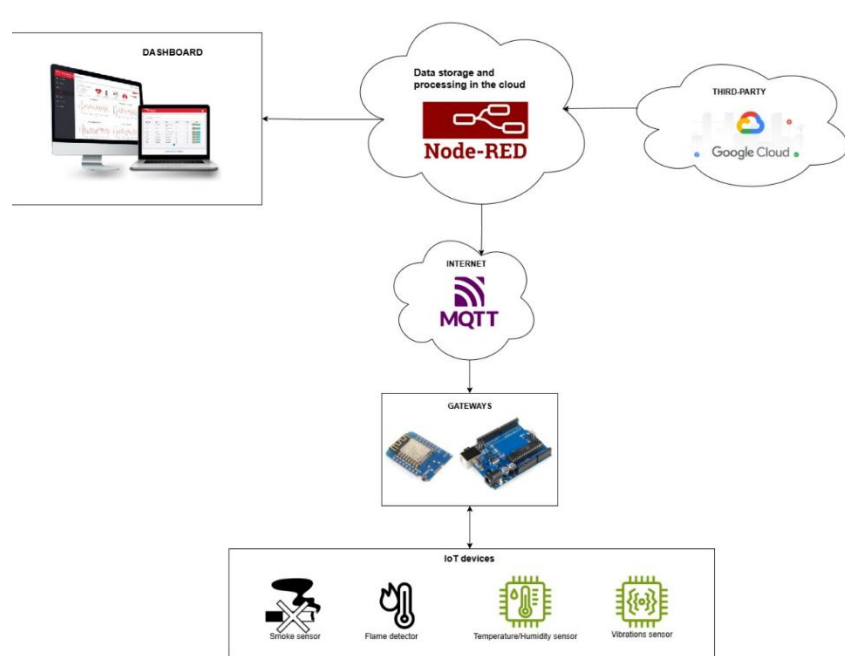


Figure 1: Communication Architecture

5. Flow Chart

The flowchart outlines the system's decision-making process for monitoring and responding to environmental and operational parameters. It continuously checks for smoke, temperature, and flame presence, while also analysing propeller speed and vibration. Based on sensor inputs, the system activates alerts and adjusts propeller control dynamically to ensure safe and efficient operation of the electric vessel.

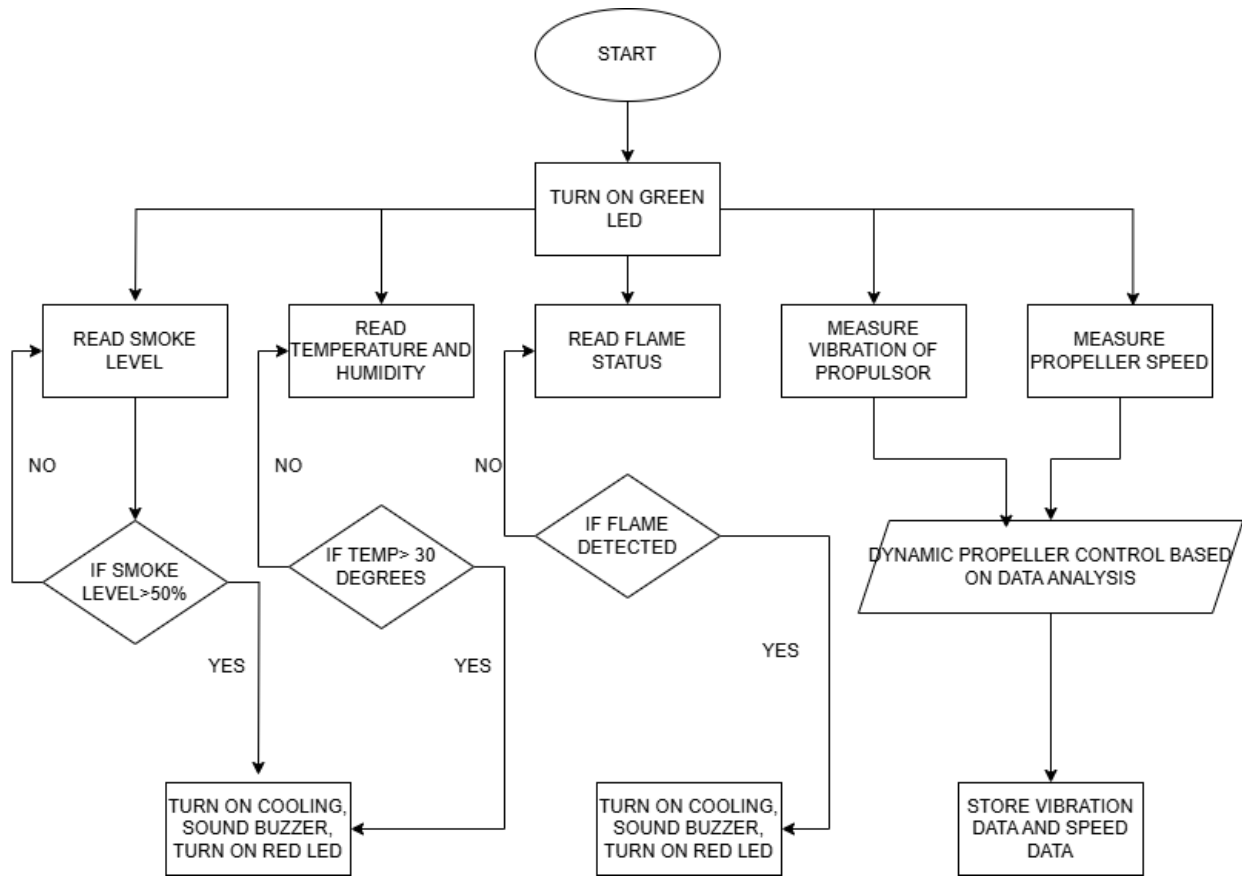


Figure 2: System Flow Chart

6. System Connection

The circuit diagram shows the physical layout of the health monitoring system using an ESP8266 microcontroller. It integrates multiple sensors—including DHT11 (temperature/humidity), flame sensor, smoke sensor, and hall effect sensor—with a 16x2 I2C LCD, buzzer, and LED indicators. All components are connected via a breadboard, demonstrating a compact and modular design for real-time monitoring and alerting.

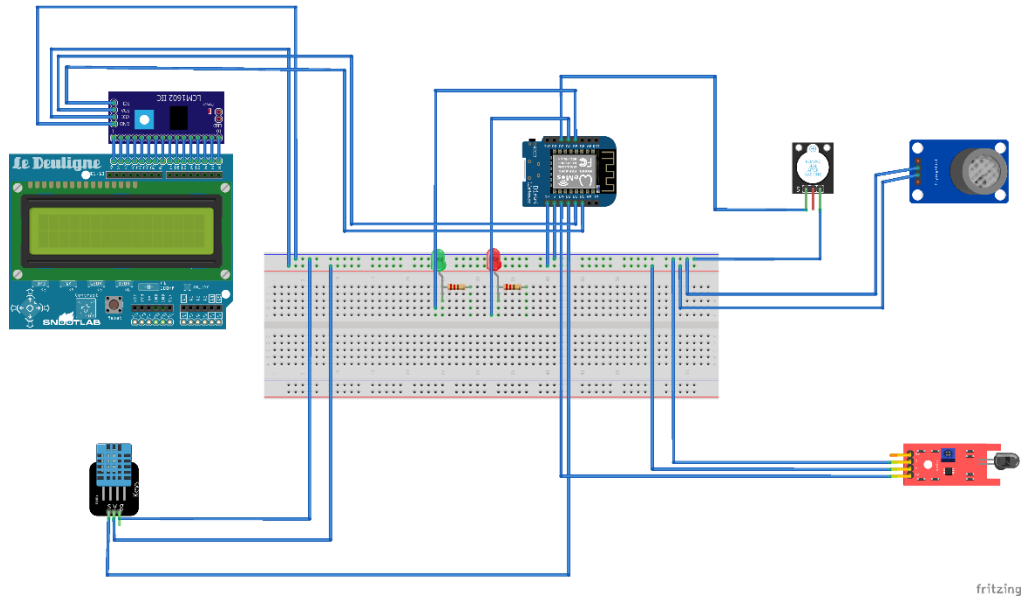


Figure 3: Sensors and Actuators Connection

Test and Validation

To validate the functionality and reliability of the hybrid ship propulsion management and health monitoring system, a series of tests were conducted under controlled simulated conditions. These tests focused on real-time sensor feedback, system responsiveness, automated control actions, and interface performance.

1. Sensor Feedback Accuracy

The system was tested for accurate real-time readings from the environmental and mechanical sensors:

- **Temperature & Humidity Sensors:** Values remained consistent and within expected range, showing stable readings of $\sim 26^{\circ}\text{C}$ and 67–72% humidity, as seen in both the console logs and graphical dashboard dials.
- **Smoke Sensor:** Registered a stable 12–15% during normal operation, which triggered no alerts, indicating correct threshold behavior.
- **Flame Detection:** A simulated flame event successfully triggered a fire alert. This was reflected in both the console logs (FIRE detected! SPRINKLER ACTIVATED) and the dashboard's Fire Alert indicator, confirming system reactivity and threshold sensitivity.

```

18:58:21.356 -> ✅ Status: All normal.
18:58:22.399 -> Smoke: 12%      RPM: 0   Flame: No Fire
18:58:22.399 -> Temperature: 26.00
18:58:22.399 -> Humidity: 67.00
18:58:22.439 -> ✅ Status: All normal.
18:58:23.452 -> Smoke: 12%      RPM: 0   Flame: No Fire
18:58:23.452 -> Temperature: 26.00
18:58:23.452 -> Humidity: 67.00
18:58:23.498 -> ✅ Status: All normal.
18:58:24.509 -> Smoke: 12%      RPM: 0   Flame: No Fire
18:58:24.509 -> Temperature: 26.00
18:58:24.509 -> Humidity: 68.00
18:58:24.581 -> ✅ Status: All normal.
18:58:25.589 -> Smoke: 12%      RPM: 0   Flame: FIRE
18:58:25.589 -> Temperature: 26.00
18:58:25.589 -> Humidity: 68.00
18:58:25.631 -> ALERT: FIRE detected! SPRINKLER ACTIVATED
18:58:26.641 -> Smoke: 12%      RPM: 0   Flame: FIRE
18:58:26.641 -> Temperature: 26.00
18:58:26.641 -> Humidity: 68.00
18:58:26.722 -> ALERT: FIRE detected! SPRINKLER ACTIVATED
18:58:27.695 -> Smoke: 12%      RPM: 0   Flame: FIRE
18:58:27.695 -> Temperature: 26.00

```

Figure 4: Serial monitor display

2. Vibration and RPM Monitoring

- **Propulsion System Readings:** The system-maintained RPM stability around 1580–1640 RPM on both left and right engines. The dials on the dashboard visually confirmed real-time tracking and motor control.
- **Vibration Analysis:** The system displayed clear spikes in vibration when disturbances were introduced. The vibration graph recorded a significant spike followed by a return to baseline, validating the sensor's ability to detect and log anomalies. These events were logged under esp8266/vibration, indicating accurate communication and storage of time-series data.

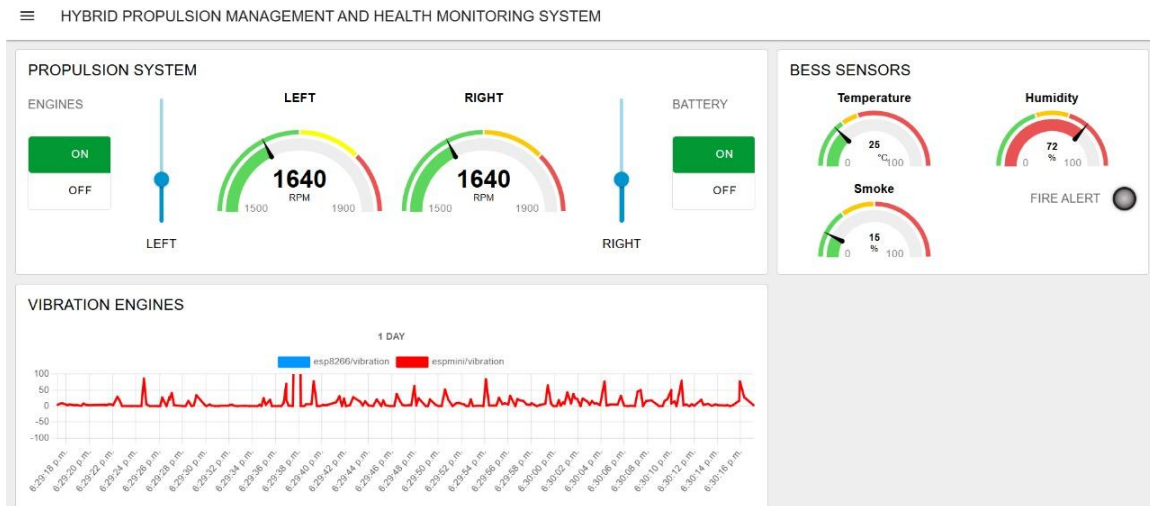
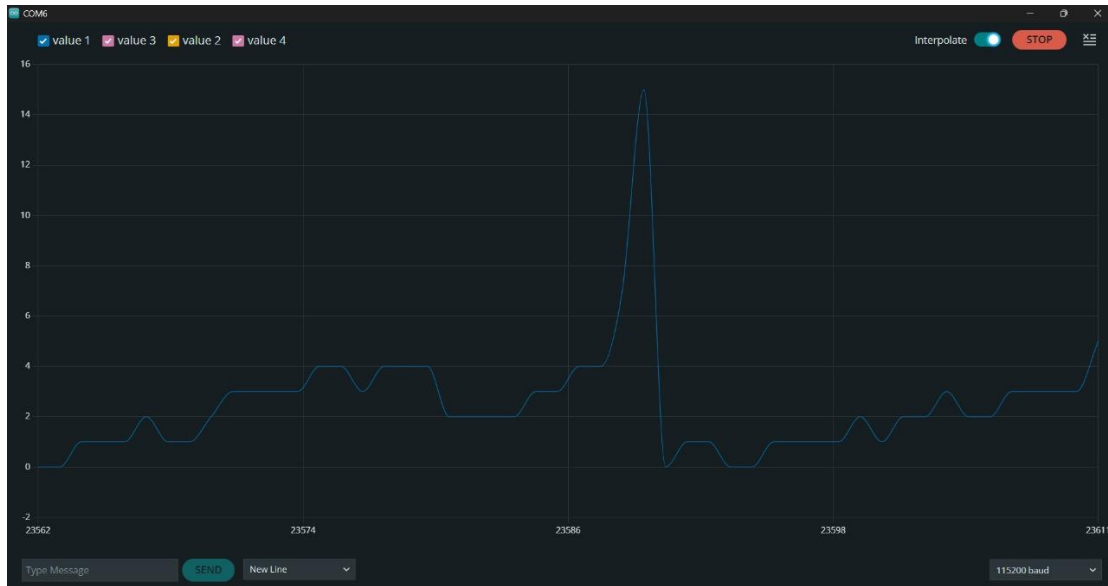


Figure 5: Flowfuse Dashboard

3. Automated Response System

One of the core validations involved testing how the system responded to abnormal conditions:

- When the flame sensor was triggered:
 - Console logs immediately switched from "Status: All normal" to "ALERT: FIRE detected! SPRINKLER ACTIVATED".
 - No RPM fluctuation occurred, ensuring the propulsion system remained isolated from the fire suppression logic.
 - The Fire Alert indicator on the dashboard activated, providing visual confirmation.

These consistent triggers demonstrated proper execution of predefined response protocols.

4. User Interface Performance

The web-based dashboard operated in real-time and offered clear visualizations of key data:

- RPM gauges responded immediately to simulated motor changes.
- BESS sensor gauges for smoke, temperature, and humidity updated smoothly.
- Fire alert functionality was integrated seamlessly, enhancing situational awareness.

Additionally, console-based logs ran in parallel with the GUI, confirming that backend processes and UI layers were in sync.

Summary of Tests

Feature Tested	Expected Outcome	Result
Temperature/Humidity	Stable live readings	✔ Passed
Smoke Detection	Alert at >50% threshold	✔ Passed
Flame Detection	Instant fire alert and sprinkler activation	✔ Passed
RPM Tracking	1500–1900 RPM with stable feedback	✔ Passed
Vibration Monitoring	Spike detection and logging	✔ Passed
Dashboard UI Sync	Live update, status indicators	✔ Passed

The test outcomes confirm that the system performs effectively under normal and abnormal conditions, delivering accurate monitoring, timely alerts, and appropriate automated responses. Future validation will include testing under mobile/remote access scenarios and introducing randomized failure simulations to stress-test system limits.

7. ETHEREUM SMART CONTRACT

7.1. Introduction

In the maritime industry, engine room safety is critical to preventing catastrophic accidents. Traditional systems rely on human intervention to react to problems such as fire, overheating, or electrical failures. This project proposes the development of a smart contract on Ethereum, which will automate monitoring, record data on the blockchain and activate emergency measures in case of risk, reducing the possibility of negligence on the part of the crew.

7.2. Case definition

The system will monitor in real time the following critical parameters within the engine room:

- Temperature: Risk of overheating.
- Humidity: Possibility of electrical short circuits.
- Smoke: Indication of serious failure or incipient fire.
- Fire: Confirmation of an emergency.
- Engine Revolutions Per Minute (RPM): Evaluation of prolonged wear and tear at full power.

If the system detects prolonged dangerous conditions without corrective action by the crew, negligence will be recorded on the blockchain, affecting potential insurance claims. In addition, the smart contract will be able to shut down engines or cut off the battery system in extreme cases to prevent further damage.

7.3. Motivation

The motivation behind this project lies in:

- Reduce security risks through automation.
- To avoid economic losses arising from avoidable damages.
- Create verifiable evidence on blockchain about negligence or emergency response.
- To offer a reliable database for insurers and boat owners.

7.4. Objectives

Design a stand-alone smart contract that records sensor information, assesses negligence and activates safety measures in a ship's engine room.

- Monitor temperature, humidity, smoke, and fire with permanent records on blockchain.
- Detect prolonged use of maximum RPMs and apply automatic fines.
- Determine negligence based on failure to respond to safety alerts.
- Allow the consultation of records by insurers to validate insurance payments.
- Turn off motors and battery systems in critical situations to prevent further damage.

7.5. Design Features

The smart contract will contain the following key functionalities:

- Automatic sensor monitoring with blockchain data storage.
- Detection of negligence if the crew does not react to critical warnings.
- Automated safety system that shuts down motors and batteries in extreme cases.
- Blockchain data logging to ensure integrity and prevent tampering.
- Access for insurers who will be able to consult the history of incidents.

The contract will run on Solidity and be deployed on the Sepolia network, an Ethereum testnet.

7.6. Implementation

Technologies used:

- Solidity: Development of the smart contract.
- Node-RED + MQTT: Receiving sensor data from the ship.
- MetaMask: Transaction signing and execution on blockchain.
- Alchemy: Smart contract connection to Ethereum.

System Flow:

- The sensors send MQTT data to Node-RED.
- Node-RED formats the information and signs it with MetaMask.
- The signed transaction is sent to Sepolia via Alchemy/Infura.
- The smart contract evaluates the data and decides if there was negligence.
- If the risk is high, the contract shuts down motors and batteries automatically.
- The records are stored on the blockchain for insurance companies.

7.7. Test & Validation

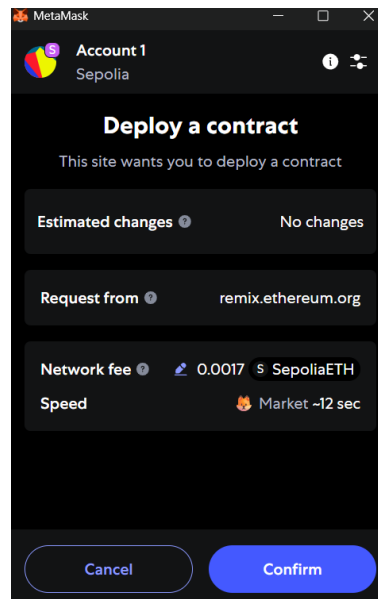
To validate the operation of the system, real sensor data will be sent via MQTT and evaluated in the smart contract on Ethereum.

The sensors will measure in real time:

- Temperature: Any overheating will be recorded.
- Humidity: Risk of short circuits will be identified.
- Smoke: The persistence of smoke in the engine room will be analyzed.
- Fire: Will be logged if a fire alert is triggered.
- RPM of the motors: The operating time at maximum power will be evaluated.

Validation Process:

- The sensor data will be sent to Node-RED via MQTT.
- Node-RED will format and sign the transaction with MetaMask.
- The transaction will be sent to Sepolia to be evaluated in the smart contract.
- The smart contract will decide whether to apply a fine or activate security measures.
- The final state will be reflected in the blockchain and can be consulted from Remix.



Scenario 1. The temperature was updated to obtain values above the limits. Therefore, the contract detects a risk and shuts down the motors and batteries until the problem is resolved.

A screenshot of the Node-RED interface. On the left, there's a sidebar with 'Deploy' and 'At Address' buttons. Below, 'Deployed Contracts' shows a contract named 'MONITORING AT 0X02D...C88' with a balance of 0 ETH. A list of controls includes 'grantInsurerA...', 'updateFlame' (set to FIRE), 'updateHumid...' (set to 90), 'updateMotor...' (set to 1000 RPM), 'updateSmoke' (set to 60), 'updateTempe...' (set to 60), and 'batteriesActive'. The main workspace shows a JavaScript code snippet for 'evaluateRisk' that checks for fire conditions and triggers alerts and shutdowns. Below the code, the 'decoded output' and 'logs' are displayed, showing three events: 'Alert', 'Engines shut down due to overheating', and 'Batteries deactivated due to fire risk'.

Then the contract evaluates how long the situation had like this and determines if it was an accident or negligence of not attending to the emergency.

CALL	[call] from: 0xAA714eaBAa5Bf0E31f60B95a4614DF542C7a3767 to: Monitoring.negligenceDetected() data: 0xe68...db601
from	0xAA714eaBAa5Bf0E31f60B95a4614DF542C7a3767
to	Monitoring.negligenceDetected() 0x02d4605f7A25364ad74B158bBd88ff3f8b3C888a
input	0xe68...db601
output	00000000000000000000000000000001
decoded input	{}
decoded output	{ "0": "bool: true" }
logs	[]
raw logs	[]

Scenario 2. When the engines spend a long time running at maximum capacity, at their maximum revs. This causes rapid wear and tear on the engines and a fine is applied.

85	// Evaluate whether the engines operated too long at maximum RPM
86	function checkEngineOveruse() internal { infinite gas
87	if (Left_Prop_RPM > rpmLimit && Right_Prop_RPM > rpmLimit) {
88	fineIssued = true;
89	emit FineIssued("Penalty applied for prolonged use of maximum RPM");
90	}
91	}
92	

to	Monitoring.updateMotorData(uint256,uint256) 0x02d46d5f7a25364ad74b15bbbd88ff3f8b3c888a
gas	36421 gas
transaction cost	36059 gas
input	0x60a...0076c
decoded input	{ "uint256 leftRPM": "1900", "uint256 rightRPM": "1900" }
decoded output	-
logs	[{ "from": "0x02d46d5f7a25364ad74b15bbbd88ff3f8b3c888a", "topic": "0xea927c170d07929390f076db2b2737556b092e50ef24ede05beaa402af0f5bed", "event": "FineIssued", "args": { "0": "Penalty applied for prolonged use of maximum RPM" } }]

8. Conclusions

This blockchain monitoring system provides a reliable and transparent solution for risk assessment in a ship's engine room. By automating the response to dangerous events, the dependence on the crew is reduced, avoiding human error and possible negligence.

The ability of insurers to consult allows informed decisions to be made on the payment of compensation in the event of damages. In addition, integration with Node-RED and physical sensors makes it possible to connect seamlessly with the ship's systems in real time.

9. Lessons Learned

- Integrating physical sensors with blockchain is feasible and improves data reliability.
- Automating safety measures can prevent damage before it occurs.
- Detecting crew negligence provides valuable information for insurers.

10. Future Improvements/Recommendations

- Contract optimization to reduce gas costs on the blockchain.
- Integration with artificial intelligence for predictive fault analysis.
- Using programmable insurance contracts (parametric insurance) for automatic payments.